Real-Time Object Detection Using YOLOv8 Small

1. Project Overview

This project implements real-time object detection using the YOLOv8 Small pretrained model and OpenCV. The application captures video from a laptop webcam, detects supported objects in each frame, and displays the results in a live window.

The system also displays object names, detection confidence scores, the total number of detected objects, per-class counts, and approximate frames per second (FPS). The annotated video is saved locally for later review.

2. Objectives

- Implement real-time object detection using a pretrained deep learning model.
- Capture and process live webcam frames.
- Visualize detected objects using bounding boxes and class labels.
- Display confidence scores and count detected objects.
- Record the annotated output video.

3. Technologies Used

Python: Application programming language.

Ultralytics YOLOv8: Pretrained object detection model.

OpenCV: Webcam capture, image display, and video recording.

COCO dataset classes: Standard object categories supported by the pretrained model.

4. Features

- Live object detection through a laptop webcam.
- Bounding boxes around detected objects.
- Object names and confidence scores.
- Total object count for each frame.
- Per-class object counts for up to five displayed categories.
- Approximate live FPS display.
- Annotated video recording in MP4 format.
- Keyboard control to stop detection.

5. System Requirements

- Windows computer or compatible Python environment.
- Python installed and accessible through the py command.
- Laptop webcam or compatible USB camera.
- Internet connection for the initial model download.
- Sufficient processing power to run real-time inference.
- A GPU is not mandatory, although performance depends on the computer's hardware.

6. Installation

Install the required Python packages:

```
py -m pip install ultralytics opencv-python
```

Save the application source code as `live_detection.py`.

The program uses the `yolov8s.pt` model. Ultralytics downloads the pretrained weights automatically when they are not already available locally.

7. Execution

Open PowerShell and navigate to the project directory:

```
cd "C:\Users\ryhan\Desktop\projects\opencv\live_detection"
```

Run the application:

```
py live_detection.py
```

Once the webcam opens, position objects in front of the camera. The application displays the detections and associated information in real time.

Press **Q** to stop the application and close the display window.

8. Detection and Counting

For each captured frame, the application performs the following operations:

- Reads an image frame from the webcam.
- Passes the frame to the YOLOv8 Small model
- Detects objects above the configured confidence threshold.
- Draws bounding boxes, class labels, and confidence scores.
- Counts detected instances by class.
- Displays the total count and approximate FPS.
- Writes the annotated frame to the output video.

The confidence threshold is configured to `0.35`. Detections below this threshold are excluded from the displayed results.

9. Output

The application generates an annotated video named:

```
live_detection_output.mp4
```

The output includes the detected objects, bounding boxes, class labels, confidence scores, object-count information, and FPS display.

The video is stored in the working directory from which the program is launched.

10. Model Information

The application uses YOLOv8 Small (yolov8s.pt), a pretrained object detection model from Ultralytics.

The standard pretrained model supports 80 COCO object categories, including people, cars, bicycles, bottles, chairs, laptops, and mobile phones.

Detection results depend on image quality, lighting, object visibility, camera conditions, and hardware performance.

11. Limitations

- The model may fail to detect small, obstructed, or poorly illuminated objects.
- The FPS varies according to the hardware and camera resolution.
- The object counter counts detections in individual frames; it does not track unique objects across time.
- Industrial components and other custom objects may require a separately trained model.
- The recorded video is subject to the camera's actual frame rate and processing performance.

12. Future Improvements

- Add object tracking across consecutive frames.
- Train a custom model for industrial components and machinery.
- Add detection alerts and event logging.
- Optimize inference speed and camera settings.
- Develop a graphical user interface for easier operation.

13. Conclusion

This project demonstrates the practical implementation of real-time computer vision using a pretrained YOLOv8 model and OpenCV. It combines webcam capture, object detection, visual annotation, object counting, and video recording in a single Python application.

The project provides a foundation for further exploration of computer vision, intelligent monitoring, and industrial object detection.

14. Author

Shaik Ryhan

Project: Real-Time Object Detection Using YOLOv8 Small